TrumpQuotes — OAuth & MCP

How the Claude AI connector works, end to end. Open in browser → File → Print → Save as PDF.

---

1. The Big Picture

There are two systems working together here:

- **MCP (Model Context Protocol)** — a standard that lets AI assistants call tools on external servers. Claude uses it to query subscription data, manage billing, and fetch quotes — all live from your database.
- **OAuth 2.1** — a standard for securely linking two accounts. It lets a TrumpQuotes user say "yes, I authorise Claude to access my account" without giving Claude their password.

Neither system exists without the other here. OAuth gives Claude a secure token that proves who the user is. MCP gives Claude a structured way to call your server using that token.

> **ANALOGY**
>
> Think of a hotel key card system. OAuth is the front desk that checks your ID and hands you a key card. MCP is the door reader that accepts the key card and lets you in. The front desk doesn't know about the door; the door doesn't know about your ID — they only share the key card.

2. What is MCP?

MCP (Model Context Protocol) is an open standard created by Anthropic. It lets AI assistants connect to external "tools" — functions they can call to get real data or perform actions.

Without MCP, Claude only knows what was in its training data. With MCP, Claude can call *your* server and get live information or take real actions.

How a tool call works

1. **User types a message**
   "Cancel my subscription."
2. **Claude reads the tool list**
   It knows about `cancel_subscription` because your server declared it.
3. **Claude calls the tool**
   It sends a POST request to `/api/mcp` with an Authorization header.

4 **Your server responds**

It validates the token, calls the Stripe API, updates Supabase, and returns a confirmation.

5 **Claude replies to the user**

"Done — your subscription is cancelled. You keep access until June 3rd, 2026."

The MCP endpoint: `/api/mcp`

This is a single Next.js route that speaks the MCP protocol. It handles GET, POST, and DELETE — all three are needed by the MCP spec for the Streamable HTTP transport. Your server creates a new `McpServer` instance per request (stateless), connects it to a `WebStandardStreamableHTTPServerTransport`, and returns the result.

> **WHAT "STATELESS" MEANS HERE**
>
> Each request is independent. There is no server-side session between requests — every call re-validates the token and re-queries the database. This is fine for Vercel (serverless) and keeps things simple.

3. MCP Tools

Your server exposes six tools. All tools require a valid OAuth Bearer token — if the user is not authenticated, every tool returns "Not authenticated." before doing anything.

Tool name	What it does	Subscription required?
`get_subscription_info`	Returns the user's current subscription status and the date the current billing period ends. Queries `subscriptions` directly.	No — works for any authenticated user including those without a subscription.
`get_cancellation_status`	Checks whether the subscription is currently scheduled to cancel at the end of the billing period. Returns the scheduled end date if cancellation is pending, or confirms the subscription will auto-renew.	No — works for any authenticated user (returns "No subscription found" if none exists).
`cancel_subscription`	Sets `cancel_at_period_end: true` on the Stripe subscription, then updates the DB status to `canceling`. The user keeps access until the period ends. Rejects if status is not `active`.	Yes — must have an `active` subscription.
`undo_cancellation`	Reverses a pending cancellation by setting `cancel_at_period_end: false` on Stripe and restoring DB status to `active`. Rejects if status is not `canceling`.	Yes — must have a `canceling` subscription.
`subscribe`	Finds or creates a Stripe customer for the user, creates a Checkout session, and returns the URL for the user to open	No — intended for users without a subscription.

	in their browser to complete payment. Rejects if an active subscription already exists.	
`get_quote`	Fetches a random Trump quote from the `whatdoestrumpthink.com` API and returns it. Rejects with a subscribe prompt if the user's status is not `active` or `canceling`.	Yes — must have an `active` or `canceling` subscription.

> **WHY DOES `GET_QUOTE` ALLOW `CANCELING`?**
>
> A `canceling` subscription is still paid and active — the user has just scheduled it to end at the next billing date. They have paid for the current period and should keep full access until it lapses.

4. What is OAuth 2.1?

OAuth is a protocol that lets a user give a third-party application limited access to their account on another service — without sharing their password.

In this case:

- **Resource owner** = the TrumpQuotes user
- **Client** = Claude (the application requesting access)
- **Authorization server** = your TrumpQuotes server (issues tokens)
- **Resource server** = your TrumpQuotes server (the `/api/mcp` endpoint)

The user never gives Claude their password. Instead, they go through a login & consent flow on your site, and your site gives Claude a token it can use going forward.

PKCE (Proof Key for Code Exchange)

OAuth 2.1 requires PKCE. This prevents a specific attack where someone intercepts the auth code before it reaches Claude. Here's how it works:

1. Before the flow starts, Claude generates a random secret: the **code verifier**.
2. It hashes it with SHA-256 to get the **code challenge**.
3. It sends only the *hash* to your authorization endpoint (safe to expose).
4. When exchanging the code for a token, Claude proves it holds the original secret by sending the **code verifier**.
5. Your server hashes it again and checks it matches. If someone intercepted the code, they can't use it because they don't have the verifier.

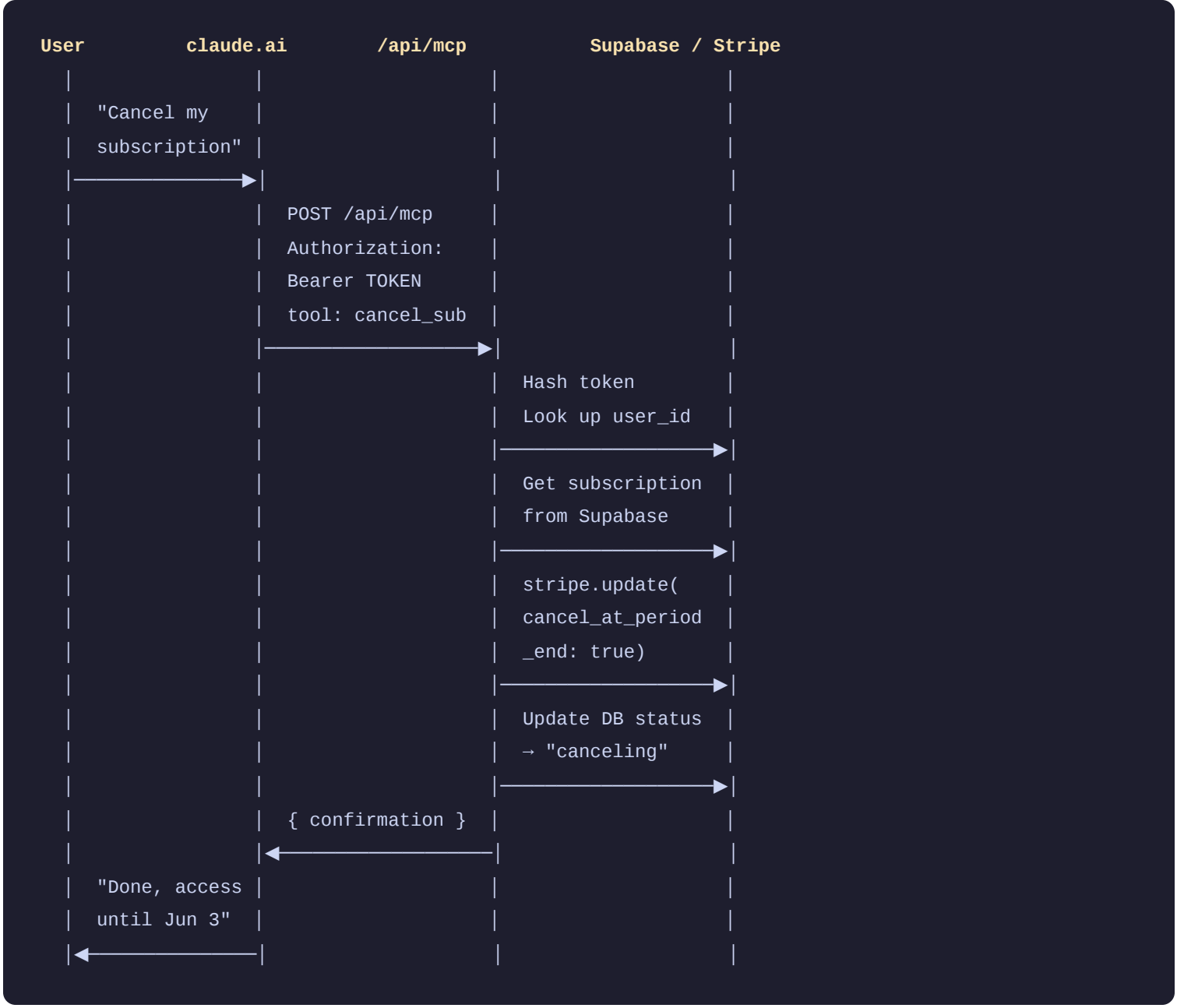
5. The Complete Auth Flow

This is what happens the first time a user adds the TrumpQuotes connector in claude.ai:


User
claude.ai
TrumpQuotes Server
Supabase DB
Add connector (URL + creds)
GET /api/mcp (no token yet)
401 + WWW-Auth resource_metadata
GET /.well-known/ oauth-protected-resource
{ authorization_servers: [base] }
GET /.well-known/ oauth-authorization-server
{ auth_endpoint, token_endpoint }
Browser opens
GET /oauth/authorize ?client_id=X&code_challenge=Y
Check session
Show consent page (logged in)
Click "Allow"
POST /api/oauth/authorize
Store auth_code hash (5 min TTL)
Redirect to claude.ai/callback ?code=RAW_CODE&state=Z

If the user clicks **Deny** instead, the browser POSTs to `/api/oauth/reject`, which redirects back to Claude's callback URL with `?error=access_denied&state=...`.

From this point on, every time the user invokes a tool:



6. OAuth Endpoints

Route	Method	What it does
`/.well-known/oauth-protected-resource`	GET	RFC 9728 resource metadata. Returns `{ resource: "/api/mcp", authorization_servers: [base] }`. Claude fetches this after receiving a 401 with a `WWW-Authenticate: Bearer resource_metadata=` header. It tells Claude which auth server to use.
`/.well-known/oauth-authorization-server`	GET	RFC 8414 auth server metadata. Returns the authorization and token endpoint URLs, supported grant types, and PKCE methods. Claude fetches this to discover the full OAuth flow automatically — no manual config needed.

`/oauth/authorize`	GET	The consent page. Claude redirects the user's browser here. Validates `client_id`, checks Supabase session (redirects to `/login?next=...` if not logged in), and shows the Allow / Deny card with the TrumpQuotes and Claude icons.
`/api/oauth/authorize`	POST	Handles the "Allow" form submission. Generates a random one-time code, hashes it with SHA-256, stores the hash + user_id + redirect_uri + PKCE challenge in `oauth_auth_codes` (5-minute TTL). Redirects the browser to Claude's callback URL with the raw code.
`/api/oauth/reject`	POST	Handles the "Deny" form submission. Redirects to Claude's callback URL with `?error=access_denied&state=...` per the OAuth spec. No database writes.
`/api/oauth/token`	POST	Server-to-server token exchange (Claude calls this directly, not via browser). Validates client credentials (`client_secret_post` or HTTP Basic), verifies PKCE, checks code expiry and redirect URI match, marks code as used. Issues a 30-day access token stored as a hash in `oauth_access_tokens`.
`/api/oauth/connections`	GET	Returns a list of active OAuth connections (access tokens) for the currently logged-in user. Used by the settings page to display connected assistants. Returns `[{ id, created_at, client_name }]`.
`/api/oauth/connections`	DELETE	Deletes a specific access token by ID, scoped to the current user. Used by the "Disconnect" button in settings. Revoking a token means Claude's next MCP call will return 401 and the user will need to reconnect.
`/api/mcp`	GET/POST/DELETE	The MCP server. On 401, returns a `WWW-Authenticate: Bearer resource_metadata=` header to trigger OAuth discovery. On valid token, exposes all six tools (see Section 3). Uses `WebStandardStreamableHTTPServerTransport` — stateless, compatible with Vercel.

7. Database Tables

`oauth_auth_codes`

Temporary one-time codes. Created when the user clicks Allow, consumed when Claude exchanges it for a token. Expire after 5 minutes and are marked `used = true` after exchange — preventing replay attacks.

Column	Type	Purpose

`code_hash`	text	SHA-256 of the raw code. Raw code is only in the redirect URL, never stored.
`user_id`	uuid	Which TrumpQuotes user approved the request.
`redirect_uri`	text	Where to send the code. Verified on token exchange to prevent open redirect attacks.
`code_challenge`	text	The PKCE hash. Verified against the code_verifier on token exchange.
`expires_at`	timestamptz	5 minutes after creation.
`used`	boolean	Prevents the same code being exchanged twice.

`oauth_access_tokens`

Long-lived tokens (30 days) issued after a successful OAuth flow. Each token is tied to one user. When Claude makes an MCP request, it sends the raw token; your server hashes it and looks it up here to find the user.

Column	Type	Purpose
`token_hash`	text	SHA-256 of the raw token. Raw token is only ever sent over HTTPS, never stored.
`user_id`	uuid	The TrumpQuotes user this token belongs to.
`client_name`	text	Hostname extracted from the `redirect_uri` at token exchange time (e.g. `claude.ai`). Displayed in the settings page next to the connection date.
`created_at`	timestamptz	When the token was issued. Displayed in the settings page as the connection date.

WHY ARE TOKENS HASHED?

If someone ever read your database (a breach, a misconfigured query, etc.), they would only see SHA-256 hashes — useless without the original values. The raw token only exists in two places: the HTTP response when it was issued, and Claude's internal storage. Your server never sees it again after hashing it.

WHY NO RLS POLICIES ON THESE TABLES?

Both tables are accessed exclusively via the Supabase **admin client** (service role key), which bypasses RLS entirely. However, RLS is still *enabled* with no policies — this means the anon key and user-scoped keys cannot access these tables at all, even accidentally. It's a lockout layer for non-admin access.

Automatic cleanup with pg_cron

Used and expired auth codes are cleaned up automatically every 10 minutes via Supabase's built-in `pg_cron` extension. This prevents the table from growing indefinitely. Codes are already rejected by application logic when expired or used — this is purely housekeeping.

To set it up, run this once in the Supabase SQL editor:

```
-- Enable the extension
create extension if not exists pg_cron;

-- Schedule cleanup every 10 minutes
select cron.schedule(
  'cleanup-oauth-auth-codes',
  '*/10 * * * *',
  $$
    delete from oauth_auth_codes
    where used = true
       or expires_at < now();
  $$
);
```

View the job: `select * from cron.job;`

Remove it: `select cron.unschedule('cleanup-oauth-auth-codes');`

View the cron jobs in the Supabase dashboard under **Database → Cron Jobs**.

8. Security Properties

Attack	How it's prevented
Someone steals the auth code from the redirect URL	PKCE: they can't exchange the code without the code verifier, which only Claude holds
Auth code used twice	Codes are marked `used = true` on first exchange and rejected on reuse
Auth code used after expiry	5-minute TTL enforced on every token exchange; stale codes purged by pg_cron
Unknown application impersonates Claude	Client ID + Secret validated on every token exchange
Token redirected to attacker's server	Redirect URI stored in the auth code and verified to match on exchange
Database breach exposes tokens	Only SHA-256 hashes stored — raw tokens never written to the database
Unauthenticated MCP access	Every MCP request validates the token before any tool runs; 401 triggers OAuth re-discovery
MCP tool called by wrong user	The token is tied to a specific `user_id`; all Supabase and Stripe queries are scoped to that ID
User revokes access	Deleting the row from `oauth_access_tokens` (via settings → Disconnect) immediately invalidates the token

9. File Map

File	What it is
`src/app/.well-known/oauth-protected-resource/route.ts`	RFC 9728 — tells Claude which auth server to use
`src/app/.well-known/oauth-authorization-server/route.ts`	RFC 8414 — tells Claude where the auth and token endpoints are
`src/app/oauth/authorize/page.tsx`	Consent UI — icons + dotted connector + Allow / Deny
`src/app/api/oauth/authorize/route.ts`	Issues auth codes after the user clicks Allow
`src/app/api/oauth/reject/route.ts`	Redirects back to Claude with error=access_denied after Deny
`src/app/api/oauth/token/route.ts`	Exchanges auth codes for access tokens (server-to-server)
`src/app/api/oauth/connections/route.ts`	GET list / DELETE one access token — used by settings page
`src/app/api/mcp/route.ts`	The MCP server — validates token and exposes all six tools
`src/app/settings/page.tsx`	Settings page shell — auth guard + layout only
`src/app/settings/AiAccessCard.tsx`	AI Assistant Access card — lists connected assistants with Disconnect button
`src/app/settings/PasswordCard.tsx`	Update Password card
`src/app/settings/PlansCard.tsx`	Plans / billing card
`src/app/settings/DangerZoneCard.tsx`	Delete Account card
`src/app/login/AuthForm.tsx`	Updated to redirect to `?next=` after login — needed for the OAuth consent redirect

10. Environment Variables

Variable	Where used	Notes
`OAUTH_CLIENT_ID`	Authorize page, token route	Public identifier for Claude as a client. Same value for all users. Generate once with `openssl rand -hex 16`.

`OAUTH_CLIENT_SECRET`	Token route	Proves the token request is from Claude. Treat like a password — add to Vercel env vars, never commit to git. Generate with `openssl rand -hex 32`.
`STRIPE_SECRET_KEY`	MCP route (cancel, undo, subscribe tools)	Already required for the main billing flow. The MCP tools call Stripe directly to update subscriptions and create checkout sessions.
`STRIPE_WEATHER_PRICE_ID`	MCP route (`subscribe` tool)	The Stripe price ID used when the subscribe tool creates a checkout session.
`NEXT_PUBLIC_APP_URL`	MCP route (`subscribe` tool)	Used to build the success and cancel URLs for the Stripe checkout session.
